

Tuition Management System API Documentation

Overview

This is a comprehensive API for managing a tuition system with different user roles (Admin, Teacher, Student, Moderator) and class scheduling functionality. The API is built using Node.js, Express, Prisma ORM with MongoDB, and includes authentication, authorization, and validation.

Base URL

`http://localhost:3000/api`

Authentication

The API uses JWT (JSON Web Token) authentication. Include the token in the Authorization header:

`Authorization: Bearer <your-jwt-token>`

Database Models

User Roles

- **ADMIN:** Full system access
- **TEACHER:** Can manage their classes and students
- **STUDENT:** Can view their booked classes
- **MODERATOR:** Limited admin privileges with configurable access control

Core Models

- **Admin:** System administrators
- **Teacher:** Instructors with qualifications and hourly rates
- **Student:** Students with grades and parent information
- **Class:** Scheduled lessons between teachers and students
- **Moderator:** Users with limited admin privileges
- **AccessControl:** Defines moderator permissions

API Endpoints

Admin Endpoints

Register Admin

http

POST /admin/register

Request Body:

json

```
{  
  "name": "Admin Name",  
  "email": "admin@example.com",  
  "password": "Password123!",  
  "role": "ADMIN",  
  "isEmailVerified": false  
}
```

Validation Rules:

- `name`: Minimum 3 characters
- `email`: Valid email format
- `password`: At least 12 characters with uppercase, lowercase, digit, and special character
- `role`: Must be "ADMIN"

Login Admin

http

POST /admin/login

Request Body:

json

```
{  
  "email": "admin@example.com",  
  "password": "Password123!"  
}
```

Get Admin Profile

http

GET /admin/me

Headers: Authorization: Bearer <token> **Required Role:** ADMIN

Update Admin

http

PUT /admin/update

Headers: Authorization: Bearer <token> **Required Role:** ADMIN

Request Body: (All fields optional)

json

```
{  
  "name": "Updated Name",  
  "email": "newemail@example.com"  
}
```

Update Admin Password

http

PUT /admin/update-password

Headers: Authorization: Bearer <token> **Required Role:** ADMIN

Verify Email

http

PUT /admin/verify-email

Headers: Authorization: Bearer <token> **Required Role:** ADMIN

Logout Admin

http

GET /admin/logout

Headers: Authorization: Bearer <token>

Teacher Endpoints

Register Teacher

http

POST /teacher/register

Request Body:

json

```
{
  "name": "Teacher Name",
  "email": "teacher@example.com",
  "qualification": "Masters in Mathematics",
  "hourlyRate": 500,
  "password": "Password123!",
  "role": "TEACHER",
  "isEmailVerified": false
}
```

Validation Rules:

- `name`: Minimum 3 characters
- `email`: Valid email format
- `qualification`: Minimum 2 characters (optional)
- `hourlyRate`: Minimum 300
- `password`: At least 8 characters with uppercase, lowercase, digit, and special character
- `role`: Must be "TEACHER"

Login Teacher

http

POST /teacher/login

Request Body:

json

```
{  
  "email": "teacher@example.com",  
  "password": "Password123!"  
}
```

Get All Teachers

http

GET /teacher/

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Query Parameters:

- page: Page number (> 1)
- limit: Items per page (> 1)

Get Teacher Profile

http

GET /teacher/me

Headers: Authorization: Bearer <token>

Get Specific Teacher

http

GET /teacher/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Update Teacher

http

PUT /teacher/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR, TEACHER (own profile)

Delete Teacher

http

DELETE /teacher/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Get Teachers for Selection

http

POST /teacher/select

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Get Teachers with Classes

http

GET /teacher/classes

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Modify Teacher Access Control

http

PUT /teacher/:id/access-control

Headers: Authorization: Bearer <token> **Required Role:** ADMIN

Request Body:

json

```
{  
  "model": "teacher",  
  "canSeeUser": true,  
  "canAddUser": false,  
  "canDeleteUser": false,  
  "canUpdateUser": true,  
  "canSeeClass": true,  
  "canAddClass": true,  
  "canDeleteClass": false,  
  "canUpdateClass": true  
}
```

Logout Teacher

http

POST /teacher/logout

Headers: Authorization: Bearer <token>

Student Endpoints

Register Student

http

POST /student/register

Request Body:

json

```
{  
  "name": "Student Name",  
  "email": "student@example.com",  
  "parentEmail": "parent@example.com",  
  "password": "Password123!",  
  "role": "STUDENT",  
  "grade": 10,  
  "isEmailVerified": false  
}
```

Validation Rules:

- `name`: Minimum 3 characters
- `email`: Valid email format
- `parentEmail`: Valid email format (optional)
- `password`: At least 8 characters with uppercase, lowercase, digit, and special character
- `role`: Must be "STUDENT"
- `grade`: Number between 1 and 12

Login Student

http

POST /student/login

Request Body:

json

```
{
  "email": "student@example.com",
  "password": "Password123!"
}
```

Get All Students

http

GET /student/

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER

Query Parameters:

- `page`: Page number (> 1)
- `limit`: Items per page (> 1)

Get Student Profile

http

GET /student/me

Headers: `Authorization: Bearer <token>` **Required Role:** STUDENT

Get Specific Student

http

GET /student/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, TEACHER, MODERATOR

Update Student (Self)

http

PUT /student/me/update

Headers: Authorization: Bearer <token> **Required Role:** STUDENT

Update Student (Admin)

http

PUT /student/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, TEACHER, MODERATOR

Delete Student

http

DELETE /student/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Get Students for Selection

http

GET /student/select

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR, TEACHER

Get Students with Classes

http

GET /student/classes

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR

Logout Student

http

POST /student/logout

Headers: Authorization: Bearer <token> **Required Role:** STUDENT

Class Endpoints

Create Class

http

POST /class/create

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR, TEACHER

Request Body:

json

```
{
  "subject": "Mathematics",
  "scheduledAt": "2024-12-01T10:00:00Z",
  "startTime": "2024-12-01T10:00:00Z",
  "teacherId": "507f1f77bcf86cd799439011",
  "studentId": "507f1f77bcf86cd799439012",
  "classLink": "https://app.conceptboard.com/board/XXXX-XXXX-XXXX-XXXX-XXXX",
  "duration": "60",
  "classStatus": "SCHEDULED"
}
```

Validation Rules:

- subject: Minimum 3 characters
- scheduledAt: Valid datetime
- startTime: Valid datetime (optional)

- `teacherId`: 24-character MongoDB ObjectId
- `studentId`: 24-character MongoDB ObjectId
- `classLink`: Must match Conceptboard URL pattern (optional)
- `duration`: Must be greater than 40 minutes
- `classStatus`: SCHEDULED, LIVE, CANCELLED, COMPLETED, IN_PROGRESS

Get All Classes

http

GET /class/

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER, STUDENT

Query Parameters:

- `startDate`: Filter by start date
- `endDate`: Filter by end date
- `sortBy`: teacher, student, classStatus, subject, startTime, day, hour, month, grade
- `order`: asc, desc
- `studentId`: Filter by student ID
- `teacherId`: Filter by teacher ID
- `classStatus`: SCHEDULED, LIVE, CANCELLED, COMPLETED, IN_PROGRESS, all-classes
- `page`: Page number (> 1)
- `limit`: Items per page (> 1)

Get Classes for Selection

http

GET /class/select

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER

Get Classes Count by Group

http

GET /class/count-by-group

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER, STUDENT

Query Parameters:

- `groupBy`: teacher, student, status, subject, startTime, day, hour, month, grade

Get Grouped Classes

http

GET /class/group

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER, STUDENT

Get Calendar View Classes

http

GET /class/calander-view/

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER, STUDENT

Get Classes Count

http

GET /class/count

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER, STUDENT

Get Specific Class

http

GET /class/:id

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER, STUDENT

Update Class

http

PUT /class/:id

Headers: `Authorization: Bearer <token>` **Required Roles:** ADMIN, MODERATOR, TEACHER

Request Body: (All fields optional)

```
json
{
  "subject": "Physics",
  "scheduledAt": "2024-12-01T11:00:00Z",
  "classStatus": "COMPLETED"
}
```

Delete Class

http

DELETE /class/:id

Headers: Authorization: Bearer <token> **Required Roles:** ADMIN, MODERATOR, TEACHER

Class Status Enum Values

- **SCHEDULED:** Class is scheduled for future
 - **IN_PROGRESS:** Class is currently active
 - **COMPLETED:** Class has been completed
 - **CANCELLED:** Class has been cancelled
 - **LIVE:** Class is live (alternative to IN_PROGRESS)
-

Error Responses

400 Bad Request

json

```
{
  "success": false,
  "message": "Validation error",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email format"
    }
  ]
}
```

401 Unauthorized

json

```
{
  "success": false,
  "message": "Access denied. No token provided."
}
```

403 Forbidden

json

```
{
  "success": false,
  "message": "Access denied. Insufficient permissions."
}
```

404 Not Found

json

```
{
  "success": false,
  "message": "Resource not found"
}
```

500 Internal Server Error

json

```
{  
  "success": false,  
  "message": "Internal server error"  
}
```

Success Response Format

```
json  
  
{  
  "success": true,  
  "message": "Operation successful",  
  "data": {  
    // Response data  
  }  
}
```

Authentication Flow

1. **Register** a user (Admin/Teacher/Student)
 2. **Login** with email and password to receive JWT token
 3. **Include token** in Authorization header for protected routes
 4. **Use role-based access** - each endpoint specifies required roles
 5. **Logout** to invalidate the token
-

Access Control

The system implements role-based access control:

- **ADMIN:** Full system access
 - **MODERATOR:** Configurable permissions via AccessControl model
 - **TEACHER:** Access to own classes and students
 - **STUDENT:** Access to own booked classes only
-

Validation

All inputs are validated using Zod schemas:

- **Email validation:** Proper email format
 - **Password validation:** Strong password requirements
 - **MongoDB ObjectId:** 24-character validation
 - **Date validation:** ISO datetime format
 - **Enum validation:** Predefined values only
-

Environment Variables

Required environment variables:

- `MONGO_URI`: MongoDB connection string
 - `JWT_SECRET`: Secret key for JWT signing
 - `PORT`: Server port (default: 3000)
-

Dependencies

Key dependencies used:

- **Express:** Web framework
- **Prisma:** ORM for database operations
- **MongoDB:** Database
- **JWT:** Authentication
- **Zod:** Schema validation
- **bcryptjs:** Password hashing
- **CORS:** Cross-origin resource sharing